

Vertex Resource Manager Specification

Version: 1.0

Status: Draft

1. Objetivo

O Resource Manager é o subsistema do Vertex responsável por controlar, distribuir e limitar o uso de recursos do sistema entre Core, plugins e subsistemas.

Seu objetivo é garantir uso eficiente e justo de recursos como:

- CPU
- RAM
- bateria
- armazenamento
- rede
- ciclos de execução

Ele atua como camada central de budgeting e controle de recursos, garantindo estabilidade e previsibilidade do sistema.

2. Responsabilidades

O Resource Manager é responsável por:

- definir budgets de recursos por plugin e sistema;
- monitorar consumo em tempo real;
- impor limites de uso;
- distribuir recursos de forma justa;
- priorizar componentes críticos do Core;
- integrar-se com Power Management e Memory Management;
- bloquear ou reduzir consumo excessivo;
- fornecer métricas de uso;
- prever saturação de recursos;
- coordenar throttling do sistema.

3. Arquitetura

Vertex Core

↓

Resource Manager

- |— Budget Engine
- |— Resource Tracker
- |— Allocation Controller
- |— Priority Resolver
- |— Throttling Engine
- |— Quota Manager
- |— Forecast Engine
- |— Enforcement Layer
- |— Metrics Collector

↓

System Resources (CPU / RAM / Battery / IO)

4. Componentes

Budget Engine

Define limites de recursos por entidade:

- plugins;
- Core subsystems;
- UI runtime.

Resource Tracker

Monitora uso em tempo real.

Allocation Controller

Decide como os recursos são distribuídos.

Priority Resolver

Define prioridade entre consumidores de recursos.

Throttling Engine

Reduz consumo quando necessário.

Quota Manager

Garante que cada plugin respeite seus limites.

Forecast Engine

Prevê consumo futuro com base em padrões.

Enforcement Layer

Aplica restrições de forma ativa.

Metrics Collector

Coleta estatísticas de uso de recursos.

5. Fluxos

Alocação de recurso

Plugin / Core

↓

Request Resource

↓

Budget Engine

↓

Allocation Controller

↓

Resource Granted

Excesso de uso

Resource Tracker

↓

Threshold Exceeded

↓

Throttling Engine

↓

Enforcement Layer

↓

Limitation Applied

Previsão de saturação

Forecast Engine

↓

Risk Detection

↓

Budget Adjustment

↓

Preventive Throttling

6. APIs

Consulta

- getUsage()
- getQuota()
- getBudget()

Controle

- allocate()
- release()
- throttle()

Configuração

- setBudget()
- updateQuota()
- setPriority()

Monitoramento

- usageStats()
- pressureLevel()

7. Estados

Sistema de recursos

Optimal

↓

Normal

↓

High Usage

↓

Critical

↓

Throttled

↓

Recovery

Estado por consumidor

- Normal
- Limited
- Throttled
- Suspended
- Blocked

8. Políticas

Budget obrigatório

Todo componente deve ter orçamento de recursos definido.

Prioridade do Core

O Core sempre tem prioridade sobre plugins.

Fairness

Recursos devem ser distribuídos de forma justa entre plugins.

Throttling progressivo

Limitações são aplicadas de forma gradual sempre que possível.

Prevenção

O sistema deve agir antes da saturação completa.

Cooperação com outros sistemas

Resource Manager coordena com:

- Power Management;
- Memory Management;
- Scheduler;
- Execution Runtime.

9. Integrações

O Resource Manager integra-se com:

- Execution Runtime;
- Scheduler;
- Plugin Manager;
- Plugin Context;
- Memory Management;
- Power Management;
- Threading Model;
- State Engine;
- Event System;
- UI Runtime;
- Logging Framework;
- Security Framework;
- Boot Manager;
- Configuration System.

10. Métricas

O sistema coleta:

- uso de CPU por plugin;
- uso de RAM por plugin;
- consumo de bateria estimado;
- uso de IO;
- número de throttles aplicados;
- número de bloqueios;

- tempo em estado crítico;
- eficiência de distribuição;
- picos de consumo;
- previsão vs uso real.

11. Exemplos

Plugin consumindo muitos recursos

1. Plugin excede quota de CPU.
2. Resource Tracker detecta excesso.
3. Throttling Engine reduz execução.
4. Scheduler ajusta prioridade.
5. Plugin continua em modo limitado.

Sistema em carga alta

1. Muitos plugins ativos.
2. Forecast Engine prevê saturação.
3. Budget Engine reduz alocações não críticas.
4. UI mantém prioridade.
5. Background tasks são reduzidos.

Economia de bateria

1. Power Management sinaliza bateria baixa.
2. Resource Manager reduz budgets.
3. Execução em background é limitada.
4. UI permanece funcional.

Uso normal

1. Plugins requisitam recursos.
2. Budget Engine valida disponibilidade.
3. Recursos são distribuídos normalmente.
4. Sistema permanece em estado Optimal.

12. Regras Arquiteturais

- Nenhum componente pode usar recursos sem controle do Resource Manager.
- Core tem prioridade absoluta em cenários críticos.
- Throttling deve ser progressivo, não abrupto.
- Previsões devem ser usadas para evitar falhas.
- Plugins nunca acessam recursos diretamente.
- Limites devem ser respeitados pelo Execution Runtime e Scheduler.

13. Limitações

O Resource Manager não é responsável por:

- execução de código;
- gerenciamento de UI;
- armazenamento de dados;
- comunicação entre plugins;
- definição de lógica de negócios.

14. Futuras Extensões

O Resource Manager poderá evoluir para:

- alocação dinâmica baseada em IA;
- ajuste de recursos por contexto do usuário;
- otimização por aplicativo em tempo real;
- integração com hardware específico de fabricantes;
- migração de carga entre dispositivos;
- perfis de consumo personalizados por usuário.

15. Resumo

O Resource Manager é o sistema central de controle e distribuição de recursos do Vertex. Ele garante uso eficiente de CPU, memória, bateria e outros recursos críticos, evitando saturação e mantendo o sistema responsivo. Em conjunto com Memory Management, Power Management e Scheduler, forma o núcleo de estabilidade e eficiência da plataforma.